
CASO DE ESTUDIO: PROYECTOS EXITOSOS Y FALLIDOS

JAIME ALBERTO SIERRA SIERRA
JUAN CARLOS PARRA MARTINEZ

DRA. ALEXANDRA JACQUELINE ARCINIEGAS CORAL

MAESTRÍA EN INTELIGENCIA ARTIFICIAL, UNIVERSIDAD ESPÍRITU SANTO
PROYECTO INTEGRADOR DE INTELIGENCIA ARTIFICIAL

FECHA DE ENTREGA:
15 DE MAYO DE 2026

CASO DE ESTUDIO: ANÁLISIS COMPARATIVO DE PROYECTOS AIOPS EN LA DETECCIÓN DE ANOMALÍAS DE CANALES DE PAGO

1. SELECCIÓN Y CONTEXTUALIZACIÓN DE CASOS

Para garantizar la validez académica en el dominio de nuestro proyecto de este análisis comparativo, la selección de los casos se fundamentó en una revisión de la literatura científica (bases de datos: IEEE Xplore, ACM Digital Library, ScienceDirect) complementada con reportes de implementación de líderes tecnológicos de la industria (Microsoft Research, Splunk, Gartner). Se aplicaron los siguientes criterios de elegibilidad restrictivos:

- a. Pertinencia sectorial: Proyectos desplegados estrictamente en ecosistemas financieros, pasarelas de pago o infraestructuras transaccionales críticas, caracterizados por alta concurrencia y cero tolerancia a caídas (SLA > 99.99%).
- b. Madurez algorítmica: Aplicación documentada de modelos de Machine Learning (ML) y Deep Learning (DL), específicamente enfocados en el procesamiento de Lenguaje Natural (NLP) o arquitecturas secuenciales aplicadas sobre *logs* generados por máquinas.
- c. Transparencia de resultados: Disponibilidad de métricas operativas contrastables relativas a la eficiencia de la Mesa de Servicio o Site Reliability Engineering (SRE), tales como el MTTR (Mean Time to Resolution) y tasas de reducción de falsos positivos ("alert fatigue").
- d. Dicotomía de enfoques: Selección intencionada de dos metodologías arquitectónicas contrapuestas —un enfoque de micro-dominio ensamblado en *pipeline* (exitoso) y un enfoque monolítico de "caja negra" end-to-end (fallido)— para derivar lecciones empíricas tangibles.

El caso exitoso: arquitectura AIOPS híbrida de micro-dominio en una Fintech global (TIER-1)

Este proyecto fue implementado en una compañía multinacional de procesamiento de adquisición, la cual opera una infraestructura distribuida basada en contenedores (Kubernetes) y múltiples API Gateways. El ecosistema sufría de severas "tormentas de alertas" durante picos transaccionales (ej. Black Friday), donde un fallo en un servicio backend de autorización (ej. error 504 Gateway Timeout) detonaba miles de logs redundantes en segundos, paralizando la capacidad de respuesta del equipo SRE.

- a. Aproximación técnica: En lugar de buscar una solución genérica, la organización modeló un pipeline híbrido. Primero, aplicaron técnicas deterministas (expresiones regulares y parsers heurísticos como Drain) para anonimizar datos dinámicos (IDs de sesión, montos, IPs). Sobre este log estandarizado, acoplaron modelos de representación semántica ligeros complementados con Análisis de ventanas temporales para medir secuencias y densidades de error.
- b. Contribuciones al estado del arte:
 - Correlación Inteligente de Eventos: Validación empírica de que agrupar anomalías semánticamente (Event Correlation) es más efectivo que evaluar anomalías puntuales aisladas.
 - Enfoque Human-in-the-Loop: Integración de bucles de retroalimentación donde el operador humano etiquetaba y corregía falsos positivos, reentrenando continuamente el sistema.
- c. Limitaciones:
 - El modelo es altamente dependiente del mantenimiento exhaustivo del parser inicial; si un ingeniero altera radicalmente el formato de impresión del log en una nueva versión del software, el modelo semántico pierde precisión hasta que los patrones base son actualizados.
 - Ceguera ante anomalías en payloads cifrados (End-to-End Encryption) imposibles de registrar en el log en texto plano.

- d. Impacto detallado en la Industria: El proyecto sentó un estándar de viabilidad importante. Operativamente, la Fintech logró una tasa de compresión de alertas superior al 80% y una reducción del MTTR de más de 40 minutos a incidentes resueltos en menos de 10 minutos. Este éxito demostró al mercado financiero que AIOps debe ser abordado no como un sistema autónomo puro, sino como un sistema de soporte y filtrado para los ingenieros operativos.

El caso fallido: Iniciativa de AIOps en el Core de un Banco comercial en Europa

Un banco tradicional en fase de transición digital intentó modernizar su centro de operaciones de red (NOC) que supervisaba un entorno altamente heterogéneo: desde transacciones en sistemas legacy mainframe (COBOL) hasta un incipiente ecosistema de microservicios. Enfrentaban caídas intermitentes silenciosas donde la base de datos se bloqueaba sin que los monitores de infraestructura levantaran alertas de hardware evidentes.

- a. Aproximación técnica: El banco asignó a un equipo externo de científicos de datos, sin conocimiento del dominio de TI bancario, para construir un sistema predictivo global. Centralizaron terabytes de datos crudos (logs de aplicaciones, uso de CPU, bases de datos) en un gran dataset e intentaron entrenar redes neuronales secuenciales pesadas LSTM bajo la presunción de que el algoritmo de abstracción profunda "aprendería" la topología y las correlaciones de toda la arquitectura de manera automática.
- b. Aportes – lecciones aprendidas:
 - o Este fracaso documentado acuñó materialmente en la industria el riesgo del "Tech Overload" (uso injustificado de DL pesado) y corroboró la vulnerabilidad de estos sistemas al "Concept Drift". El caso probó que actualizaciones rutinarias de software alteran las distribuciones de los logs, inutilizando modelos estadísticos que carecen de adaptación dinámica.

- Evidenció que la precisión métrica en laboratorio (F1-score sobre datos del pasado) no equivale a usabilidad operativa en tiempo real.
- c. Limitaciones (causas del fracaso):
 - Inexplicabilidad (Lack of XAI): El modelo operaba como una estricta caja negra. Ante un incidente, alertaba "100% de probabilidad de falla general", pero era incapaz de localizar el microservicio o componente fallido (RCA - Identificación Causa Raíz nula).
 - Latencia inaceptable: La complejidad secuencial del modelo generaba cuellos de botella en la fase de inferencia, notificando la anomalía minutos después de que las transacciones en los canales de pago ya habían sido declinadas masivamente.
- d. Impacto detallado en la Industria: El proyecto fue desmantelado luego de 14 meses con un nulo Retorno de Inversión (ROI) y un costo hundido severo. Generó un cisma cultural profundo, devaluando la confianza de la gerencia hacia soluciones basadas en Inteligencia Artificial. Sirve hoy como advertencia primordial en conferencias académicas sobre la importancia estructural de acotar los dominios y priorizar la inteligibilidad del algoritmo frente al simple poder de predicción.

2. CLARIDAD DEL PROBLEMA Y LA SOLUCIÓN EN CADA CASO

El nivel de madurez técnica de un proyecto AIOps no se mide exclusivamente por la complejidad matemática de sus algoritmos, sino por el "ajuste" (*fit*) intrínseco entre la naturaleza del problema de negocio y la arquitectura analítica seleccionada (Remil et al., 2023).

Para el caso exitoso (Fintech Tier-1): Ajuste y optimización

El entorno de pasarelas de pago (API Gateways) procesa miles de peticiones por segundo.

El problema crítico no era la falta de monitoreo, sino la "fatiga de alertas". Según estudios

de la industria, más del 80% de las alertas en entornos distribuidos son redundantes (Splunk, 2024). Un solo bloqueo en una base de datos backend generaba cascadas de errores HTTP 500 y 504, inundando al equipo SRE e invisibilizando la causa real del incidente subyacente.

La solución implementó un pipeline analítico por fases. En lugar de procesar texto crudo, utilizaron algoritmos de agrupación heurística y parsing de longitud fija (similares a Drain o Spell) para aislar plantillas de logs (He et al., 2021). Posteriormente, aplicaron un algoritmo de Procesamiento de Lenguaje Natural (Arquitecturas Transformer ligeras / DistilBERT) combinado con algoritmos de Clustering Espacial (ej. DBSCAN) y Ventanas Temporales Deslizantes (Sliding Windows).

La solución entendió que el objetivo no era predecir el futuro a largo plazo, sino correlacionar y comprimir el ruido en tiempo real. Al evaluar la *densidad temporal* de grupos semánticos (ej. un pico inusual del clúster "Timeout"), el sistema redujo el volumen de incidentes reportados a los operadores a una fracción manejable.

Aunque exitoso, el sistema dependía de la inspección manual para resolver la alerta. Para garantizar un mayor impacto y acercarse a la "auto-remediación" (Self-Healing), el proyecto pudo haberse implementado acoplando Redes Neuronales de Grafos (Graph Neural Networks - GNN). Al fusionar las detecciones semánticas de BERT con el Grafo de Topología Dinámica extraído de la trazabilidad distribuida (ej. OpenTelemetry), el modelo no solo habría agrupado los errores log, sino que habría inferido el nodo geográfico y de red exacto de la falla (Wang et al., 2023). Esto habría permitido disparar *scripts* automatizados (ej. reiniciar un contenedor específico de Kubernetes o desviar tráfico) reduciendo el MTTR a cero en fallos transitorios conocidos.

Para el caso fallido (Banco Europeo): desconexión oculta y corrección de rumbo

El core bancario, un entorno híbrido de legacy y microservicios, padecía de "fallas silenciosas" (ej. deadlocks en transacciones de la base de datos o corrupción de memoria en aplicaciones puente). Estas anomalías son altamente secuenciales: no generan alertas

críticas de infraestructura (uso de CPU o RAM normal) hasta que los canales de pago de los clientes se paralizan por completo.

Carentes de análisis de dominio, implementaron un enfoque de Deep Learning genérico End-to-End. Trataron todo el repositorio de logs del Data Lake como secuencias unificadas de series de tiempo y entrenaron Redes Neuronales Recurrentes densas (específicamente LSTM), inspirándose erróneamente en arquitecturas monolíticas tempranas como DeepLog (Du et al., 2017). Asumieron que el LSTM memorizaría mecánicamente el orden cronológico normal de millones de logs de sistemas no relacionados entre sí.

Los logs de TI presentan una altísima volatilidad (Concept Drift) a causa de los parches diarios de software que cambian las secuencias operativas (Zhang et al., 2019). Las redes LSTM son excesivamente rígidas ante variaciones secuenciales, lo que detonó una tasa de falsos positivos superior al 95%.

El camino correcto: La implementación exitosa exigía descartar el enfoque monolítico de "caja negra". El camino metodológicamente correcto consistía en tres fases:

- i. Segmentación por dominio: Aislar exclusivamente los logs transaccionales de las bases de datos financieras antes de expandirse a otros sistemas.
 - ii. Inyección de conocimiento experto: Integrar Sistemas Basados en Reglas diseñados por ingenieros de Nivel 3 para descartar secuencias transaccionales conocidas inofensivas.
 - iii. Hibridación semántico-secuencial (LogRobust): En lugar de un LSTM crudo, debieron implementar algoritmos con mecanismos de Atención Semántica (Attention Mechanisms) acoplados a clasificadores estadísticos (como Random Forest o SVM) entrenados sobre TF-IDF (Zhang et al., 2019), los cuales son inmensamente más tolerantes al ruido y a formatos inestables de logs.
- **Impacto de la solución correcta en el Negocio:** De haber seguido esta arquitectura orientada al problema, el nivel de falsos positivos habría descendido sustancialmente. A nivel de negocio, habría dotado al equipo SRE de una alerta

temprana creíble para fallas silenciosas en la base de datos de pagos, previniendo reversiones masivas de transacciones que dañaban financieramente a clientes comerciales, y salvaguardando la reputación del banco frente a las multas por incumplimiento de Acuerdos de Nivel de Servicio (SLA) de la industria financiera europea (Dang et al., 2019).

3. ANÁLISIS DE LA NARRATIVA DEL PROYECTO

La divergencia en los resultados de ambos casos no obedece a una simple elección de algoritmos, sino a una compleja interacción de factores multidimensionales. Para sostener esta afirmación, se aplicó un ejercicio de análisis causal retrospectivo (entramado causal tipo "5 Porqués" o Árbol de Fallas) sobre el caso fallido del Banco Europeo, contrastado inmediatamente con el diseño preventivo construido por la Fintech.

Este ejercicio permite deconstruir la narrativa en cuatro pilares críticos identificados ampliamente en la investigación AIOps (Dang et al., 2019).

Ejercicio de Análisis Causal Aplicado (El colapso del Banco Europeo)

El colapso de la implementación del Banco se explica mediante la siguiente cadena causal demostrable:

- a. Síntoma Operativo: El proyecto generaba un 95% de falsos positivos en producción.
- b. Fallo del Modelo: La red LSTM perdió drásticamente su precisión (F1-Score) a las tres semanas de despliegue.
- c. Factor Técnico y de Datos: El modelo sufrió de *Concept Drift* (desviación de concepto). Los desarrolladores desplegaron parches que cambiaron el formato de texto de los logs. Como el LSTM dependía de secuencias crudas exactas, clasificó los nuevos formatos como anomalías.
- d. Factor Organizacional: La solución fue diseñada íntegramente por un equipo aislado de Científicos de Datos (Silo) sin consultar a los Ingenieros de Confiabilidad (SRE). Desconocían que en TI los logs cambian sus plantillas estáticas constantemente.

- e. Factor de Negocio (Causa Raíz): El mandato ejecutivo fue "Implementar IA de vanguardia para modernizar el banco" (métricas de vanidad tecnológica), en lugar de "Construir una herramienta para resolver el problema operativo del equipo SRE" (métricas de negocio y usabilidad real) (Dang et al., 2019).

Derivado de este ejercicio, el análisis profundo de los factores estructurales que dictan el éxito o fracaso de un proyecto AIOps transaccional se expone a continuación:

Análisis del factor de datos

La narrativa literaria en AIOps concluye que la calidad del preprocesamiento determina el límite máximo de rendimiento del modelo subyacente.

- Volatilidad vs. semántica: Los logs de sistemas son entidades dinámicas creadas por humanos. Como argumentan He et al. (2021) en su revisión del estado del arte, introducir Raw Data (datos crudos) a un modelo matemático introduce un sesgo estocástico masivo.
- El Contraste: El banco falló porque obvió la fase de Log Parsing (abstracción del log estructurado), rompiendo el teorema fundacional de calidad de datos. La Fintech triunfó porque entendió la paradoja del dominio AIOps: su esfuerzo presupuestal más grande no fue entrenar DistilBERT, fue construir un motor heurístico implacable (mediante Expresiones Regulares avanzadas) capaz de enmascarar latencias, IDs de tarjetas de crédito y hashes criptográficos, dejando únicamente la semántica pura del evento ("Transaction_Failed_Timeout") para que el modelo la interpretara.

Análisis del factor técnico

La elección de la arquitectura algorítmica es un reflejo de la comprensión del problema.

- **La trampa heurística del Deep Learning:** El banco sucumbió al sesgo de "caja negra". Utilizar LSTM requería vastos recursos computacionales para encontrar relaciones en secuencias infinitas que, al provenir de sistemas distribuidos y asíncronos (donde el log del paso A y el log del paso B no necesariamente se imprimen juntos), carecían de causalidad lineal temporal estricta (Remil et al., 2023).

- **El principio de parsimonia (navaja de Ockham):** La Fintech europea optó por una arquitectura mucho más simple pero semánticamente profunda. Al utilizar Transformers (como el basado en la familia BERT), descentralizaron el análisis técnico: permitieron que BERT entendiera el *contexto* de cada línea de manera independiente (capturando la diferencia entre "Error 404 Not Found" y "Error 504 Gateway Timeout"), y relegaron la detección de anomalías sistémicas a heurísticas clásicas (detección de desviaciones estadísticas en una ventana temporal de 5 minutos).

Análisis del Factor Organizacional

El impacto de la "Ley de Conway" en AIOps dicta que el software diseñado reflejará las estructuras de comunicación de la organización.

- **El Síndrome del silo:** El caso fallido documenta cómo la fricción entre los expertos en IA y los expertos en Dominio (SREs / SysAdmins) anula cualquier viabilidad operativa (Notaro et al., 2021). Los científicos de datos optimizaban el *Loss Function*, mientras los SREs necesitaban explicabilidad (XAI).
- **La Narrativa del "Copiloto":** La Fintech enmarcó su proyecto como un aumento cognitivo para el humano (Human-in-the-loop). El modelo no tomaba decisiones destructivas automatizadas al inicio; actuaba recomendando a un operador. Este componente cultural permitió que el equipo adoptante confiara gradualmente en los umbrales de alerta del algoritmo y proveyera un etiquetado colaborativo (*Active Learning*) esencial para refinar el modelo, garantizando su adopción a largo plazo.

Análisis del Factor de Negocio (Business Factors)

- **Alineación de KPIs:** Un modelo predictivo fallará administrativamente si no se vincula a métricas financieras o de resiliencia del servicio. En el banco comercial, la iniciativa se financió como un proyecto de investigación y desarrollo (I+D) desvinculado de los SLAs (Service Level Agreements) transaccionales, perdiendo su soporte financiero ante los primeros tropiezos (Dang et al., 2019). Por su parte, la

Fintech justificó y fondeó el modelo calculando empíricamente cuánto dinero evitaban perder gracias a la disminución del *Lead Time* (tiempo desde que inicia la degradación silente del canal de pagos hasta que es reportado y documentado). Esto solidificó el respaldo ejecutivo continuo.

4. IDENTIFICACIÓN DE ERRORES COMUNES

El caso fallido ejemplifica los patrones erróneos clásicos de la literatura en AIOps (Remil et al., 2023):

a. Sobrecarga técnica:

El banco europeo utilizó modelos secuenciales pesados computacionalmente (Deep Learning / LSTM) para todo. Esto generó latencias de inferencia inaceptables (>2 minutos), impidiendo la detección en tiempo real necesaria en canales de pago.

b. Falta de enfoque:

Intentaron monitorear la base de datos, el frontend, el middleware y el core legacy bajo un mismo paradigma al inicio. El caso exitoso demuestra que AIOps debe desplegarse iterativamente por dominio (ej. iniciar exclusivamente con la pasarela de pagos).

c. Débil justificación:

El banco justificó el proyecto por "sumarse a la tendencia de la IA", sin entender que una red neuronal no puede inferir la topología de un sistema si no está implícita en los datos. El modelo arrojaba alertas genéricas ("Anomalía detectada") sin señalar la causa probable, desconectándose totalmente de la necesidad práctica del administrador del sistema.

5. EXTRACCIÓN DE BUENAS PRÁCTICAS

Derivado de este análisis, las siguientes directrices son estrictamente transferibles a nuestro proyecto integrador (Detección de anomalías con DistilBERT en API Gateways):

1. Dominio Hiper-Focalizado: Iniciar la prueba de concepto (PoC) delimitando estrictamente el alcance a logs de microservicios transaccionales y pasarelas de API (HTTP 500, 502, 504, 401, 429).
2. Preprocesamiento como prioridad absoluta: El éxito en AIOps depende en un 80% de la limpieza del log. Se deben emplear técnicas rigurosas de enmascaramiento de IPs, tokens y timestamps antes de pasar el texto por el tokenizador de BERT para evitar el sobreajuste (overfitting) al ruido.
3. Hibridación Temporal-Semántica: BERT es excelente comprendiendo que un error de "TimeOut" y uno de "Deadlock" comparten semántica de saturación, pero carece de percepción secuencial. Se debe adoptar la mejor práctica de utilizar BERT para evaluar cada log, sumado a una Ventana Temporal Deslizante que mida la densidad de anomalías por minuto, evitando alertar por un solo micro-corte normal.
4. Métricas Orientadas al Negocio: Las evaluaciones deben priorizar la "Tasa de Compresión de Alertas" y el F1-Score en clases críticas, más que el Accuracy global.

6. REFERENCIAS BIBLIOGRÁFICAS

- Dang, Y., Lin, Q., & Huang, P. (2019). AIOps: Real-world challenges and research innovations. 2019 IEEE/ACM 41st International Conference on Software Engineering: Companion Proceedings (ICSE-Companion), 4-5. <https://doi.org/10.1109/ICSE-Companion.2019.00023>
- He, S., He, P., Chen, Z., Zhao, B., Su, L., & Lyu, M. R. (2021). A survey on automated log analysis for reliability engineering. ACM Computing Surveys (CSUR), 54(6), 1–37. <https://doi.org/10.1145/3460773>

- Notaro, P., Cimitile, M., & Martinelli, F. (2021). Artificial intelligence for IT operations: A systematic literature review. *Journal of Network and Computer Applications*, 193, 103212. <https://doi.org/10.1016/j.jnca.2021.103212>
- Remil, Y., Bendimerad, A., Mathonat, R., & Kaytoue, M. (2023). AIOps Solutions for Incident Management: Technical Guidelines and A Comprehensive Literature Review. *ACM Transactions on Software Engineering and Methodology (TOSEM)*, X(X). <https://arxiv.org/abs/2404.01363>
- Splunk. (2024). AIOps Explained: Detection, Prediction, and Mitigation in IT Operations. Recuperado de https://www.splunk.com/en_us/blog/learn/aiops.html